

20 JSON Tasks in Go

1. Сериализация структуры в JSON

Описание: Напишите программу, которая сериализует структуру Person в формат JSON и выводит результат на экран.

Структура:

```
type Person struct {  
    Name string `json:"name"`  
    Age  int  `json:"age"`  
    Email string `json:"email"`  
}
```

2. Десериализация JSON в структуру

Описание: Напишите программу, которая десериализует JSON-строку в структуру Person и выводит результат на экран.

Структура:

```
type Person struct {  
    Name string `json:"name"`  
    Age  int  `json:"age"`  
    Email string `json:"email"`  
}
```

3. Сериализация карты в JSON

Описание: Напишите программу, которая сериализует карту map[string]int в JSON и выводит результат на экран.

Пример данных:

```
data := map[string]int{  
    "apples": 5,  
    "oranges": 10,  
}
```

4. Десериализация JSON в карту

Описание: Напишите программу, которая десериализует JSON-строку в карту `map[string]int` и выводит результат на экран.

Пример данных:

```
{  
    "apples": 5,  
    "oranges": 10  
}
```

5. Пропуск полей при сериализации

Описание: Напишите программу, которая сериализует структуру, пропуская поля с пустыми значениями.

Структура:

```
type Product struct {  
    Name    string `json:"name"`  
    Price   float64 `json:"price,omitempty"`  
    InStock bool   `json:"in_stock,omitempty"`  
}
```

6. Работа с вложенными структурами

Описание: Напишите программу, которая сериализует и десериализует структуру с вложенной структурой.

Структура:

```
type Address struct {  
    Street string `json:"street"`  
    City   string `json:"city"`  
}  
  
type Person struct {  
    Name   string `json:"name"`  
    Age    int    `json:"age"`  
    Email  string `json:"email"`  
    Address Address `json:"address"`  
}
```

7. Использование JSON с интерфейсами

Описание: Напишите программу, которая сериализует и десериализует карту `map[string]interface{}` и выводит результат на экран.

Пример данных:

```
data := map[string]interface{}{  
    "name": "John",  
    "age":  30,  
    "email": "john@example.com",  
}
```

8. Чтение JSON из файла

Описание: Напишите программу, которая читает JSON из файла и десериализует его в структуру.

Файл: `person.json`

Структура:

```
type Person struct {  
    Name string `json:"name"`  
    Age  int  `json:"age"`  
    Email string `json:"email"`  
}
```

9. Запись JSON в файл

Описание: Напишите программу, которая сериализует структуру в JSON и записывает результат в файл.

Файл: output.json

Структура:

```
type Product struct {  
    Name string `json:"name"`  
    Price float64 `json:"price"`  
}
```

10. Пропуск неизвестных полей при десериализации

Описание: Напишите программу, которая десериализует JSON-строку в структуру, игнорируя неизвестные поля.

Структура:

```
type Person struct {  
    Name string `json:"name"`  
    Age  int  `json:"age"`  
}
```

11. Форматирование JSON-вывода

Описание: Напишите программу, которая сериализует структуру в JSON с отступами для улучшения читаемости.

Метод: Используйте `json.MarshalIndent`.

Структура:

```
type Person struct {  
    Name string `json:"name"`  
    Age  int  `json:"age"`  
    Email string `json:"email"`  
}
```

12. Обновление значения в JSON

Описание: Напишите программу, которая десериализует JSON в структуру, изменяет одно из значений, а затем сериализует структуру обратно в JSON.

Структура:

```
type Person struct {  
    Name string `json:"name"`  
    Age  int  `json:"age"`  
    Email string `json:"email"`  
}
```

13. Парсинг JSON с разными типами данных

Описание: Напишите программу, которая десериализует JSON-строку, содержащую данные разных типов (строки, числа, массивы, объекты), в `map[string]interface{}`.

Пример данных:

```
{  
    "name": "John",  
    "age": 30,  
    "scores": [100, 98, 95],  
    "address": {"city": "New York", "street": "5th Avenue"}  
}
```

14. Массив структур в JSON

Описание: Напишите программу, которая сериализует массив структур Person в JSON и выводит результат на экран.

Структура:

```
type Person struct {  
    Name string `json:"name"`  
    Age  int  `json:"age"`  
    Email string `json:"email"`  
}
```

15. Добавление нового объекта в JSON-массив

Описание: Напишите программу, которая читает JSON-массив из файла, добавляет новый объект и записывает обновленный массив обратно в файл.

Файл: people.json

Структура:

```
type Person struct {  
    Name string `json:"name"`  
    Age  int  `json:"age"`  
    Email string `json:"email"`  
}
```

```
}
```

16. Удаление объекта из JSON-массива

Описание: Напишите программу, которая читает JSON-массив из файла, удаляет объект с определенным значением поля и записывает обновленный массив обратно в файл.

Файл: people.json

Структура:

```
type Person struct {  
    Name string `json:"name"`  
    Age  int  `json:"age"`  
    Email string `json:"email"`  
}
```

17. Поиск объекта в JSON-массиве

Описание: Напишите программу, которая читает JSON-массив из файла и находит объект с определенным значением поля.

Файл: people.json

Структура:

```
type Person struct {  
    Name string `json:"name"`  
    Age  int  `json:"age"`  
    Email string `json:"email"`  
}
```

18. Подсчет объектов в JSON-массиве

Описание: Напишите программу, которая читает JSON-массив из файла и

подсчитывает количество объектов в массиве.

Файл: people.json

Структура:

```
type Person struct {  
    Name string `json:"name"`  
    Age  int  `json:"age"`  
    Email string `json:"email"`  
}
```

19. Десериализация JSON с произвольными полями

Описание: Напишите программу, которая десериализует JSON-строку с произвольным набором полей в `map[string]interface{}`.

Пример данных:

```
{  
    "name": "John",  
    "age": 30,  
    "country": "USA",  
    "job": "Engineer"  
}
```

20. Сериализация и десериализация времени в JSON

Описание: Напишите программу, которая сериализует и десериализует структуру с полем времени (`time.Time`) в JSON.

Структура:

```
type Event struct {  
    Name string `json:"name"`
```


Timestamp time.Time `json:"timestamp"`

}